

WINDOWS LOW LEVEL DETECTABILITY CHEAT SHEET

PE File Format Structures

PE Entity Name	C Structure Name
DOS Header	IMAGE_DOS_HEADER
DOS Stub	IMAGE_DOS_HEADER
NT Headers	IMAGE_NT_HEADERS
File Header	IMAGE_FILE_HEADER
Optional Header	IMAGE_OPTIONAL_HEADER
Section Table (Section Headers)	IMAGE_SECTION_HEADER
Export Directory	IMAGE_EXPORT_DIRECTORY
Import Directory	IMAGE_IMPORT_DESCRIPTOR
Resource Directory	IMAGE_RESOURCE_DIRECTORY
Exception Directory	IMAGE_RUNTIME_FUNCTION_ENTRY
Security Directory	WIN_CERTIFICATE
Base Relocation Table	IMAGE_BASE_RELOCATION

PE Entity Name	C Structure Name
Debug Directory	<code>IMAGE_DEBUG_DIRECTORY</code>
Architecture Directory	<code>IMAGE_ARCHITECTURE_HEADER</code>
Global Pointer Directory	<code>IMAGE_DATA_DIRECTORY</code>
TLS Directory	<code>IMAGE_TLS_DIRECTORY</code>
Load Configuration Directory	<code>IMAGE_LOAD_CONFIG_DIRECTORY</code>
Import Address Table	<code>IMAGE_THUNK_DATA</code>

PE Sections

Section Header	Description
<code>.text</code>	Contains the executable code of the program. This is the only section that should include code and can carry out execution.
<code>.rdata</code>	Holds read-only initialized data, such as literal strings, constants, and debug directory information.
<code>.data</code>	Stores initialized global and static variables that are accessible throughout the program.
<code>.pdata</code>	Contains exception handling and unwind information, supporting structured exception handling (SEH).
<code>.rsrc</code>	Houses resources used by the program, including icons, bitmaps, and other embedded files.
<code>.reloc</code>	Contains relocation information, allowing the program to be loaded at different memory addresses.
<code>.bss</code>	Contains uninitialized data, typically global and static variables that are zero-initialized at runtime.

Section Header	Description
.edata	Holds the export table, listing functions and data exported by the module.
.idata	Contains the import table, detailing functions and data imported from other modules.
.tls	Stores Thread Local Storage data, which is unique for each thread in the process.
.debug	Includes debugging information used by debuggers and development tools.
.stab	Used for symbol table information, primarily in older or specific debugging formats.
.sxdata	Supports structured exception handling (SEH) by listing exception registration data.
.bound_import	Contains information about statically bound imports to speed up the loading process.
.loadcfg	Holds load configuration data, including security features like SafeSEH and DEP settings.
.crt	Contains C runtime initialization and termination routines, such as constructors and destructors.
.rsrc\$01	A subsection of .rsrc , potentially used for specific resource types like icons or menus.
.reloc\$A	A subsection of .reloc , used for specific relocation information.
.mrdata	Stores managed read-only data, typically used in .NET assemblies to hold metadata and other read-only information.

Win32 API Essentials

API Function Name	Purpose
<u>CreateProcess</u>	Creates a new process and its primary thread, often used to launch malicious executables or inject code into processes.
<u>WriteProcessMemory</u>	Writes data to the memory of a specified process, commonly used for code injection or process hollowing.
<u>VirtualAllocEx</u>	Allocates memory in a specified process's virtual address space, often used for injecting malicious payloads.
<u>CreateRemoteThread</u>	Creates a thread in the virtual address space of another process, used to execute injected code.
<u>OpenProcess</u>	Opens an existing process for manipulation, allowing malware to access or modify a target process.
<u>ReadProcessMemory</u>	Reads data from the memory of a specified process, used to steal data or analyze process memory.
<u>GetProcAddress</u>	Retrieves the address of an exported function from a DLL, used for dynamic API resolution to evade detection.
<u>LoadLibrary</u>	Loads a DLL into the address space of the calling process, often used to load malicious DLLs.
<u>CreateFile</u>	Creates or opens a file or I/O device, used for file manipulation, logging, or dropping malicious payloads.
<u>WriteFile</u>	Writes data to a file, commonly used to create or modify malicious files on disk.
<u>RegCreateKey</u>	Creates a registry key, used for persistence (e.g., adding to startup) or storing configuration.
<u>RegSetValue</u>	Sets a value for a registry key, used for persistence or storing malicious configuration data.
<u>InternetOpen</u>	Initializes an application's use of WinINet functions, used for C2 communication or data exfiltration.

API Function Name	Purpose
<u>InternetConnect</u>	Establishes a connection to a server, used for communication with malicious servers.
<u>HttpSendRequest</u>	Sends an HTTP request, often used to communicate with a malicious server for updates or data theft.
<u>CryptEncrypt</u>	Encrypts data, used to obfuscate malicious payloads or communications.
<u>CryptDecrypt</u>	Decrypts data, used to decode malicious payloads or communications.
<u>GetSystemInfo</u>	Retrieves information about the system, used to fingerprint the environment or tailor attacks.
<u>CreateMutex</u>	Creates a mutex object, used to ensure only one instance of the malware runs or to mark an infected system.
<u>ShellExecute</u>	Executes a command or opens a file, used to launch malicious scripts or applications.
<u>FindWindow</u>	Finds a window by name or class, used for process manipulation or UI-based attacks (e.g., keylogging).
<u>SendMessage</u>	Sends a message to a window, used for UI manipulation or triggering actions in other processes.
<u>SetWindowsHookEx</u>	Installs a hook procedure to monitor system events, commonly used for keylogging or capturing user input.

DR7 Configuration for DR0–DR3

Breakpoint	Enable Bit (Lx)	RWx (Bits)	LENx (Bits)	Access Type	Description
DR0	Bit 0 (L0)	00 (16–17)	00 (18–19)	Execution	Triggers on instruction execution at the address in DR0

Breakpoint	Enable Bit (Lx)	RWx (Bits)	LENx (Bits)	Access Type	Description
DR0	Bit 0 (L0)	01 (16–17)	00 (18–19)	Write (1 byte)	Triggers on 1-byte write to the address in DR0
	Bit 0 (L0)	11 (16–17)	11 (18–19)	Read/Write (4 bytes)	Triggers on 4-byte read or write to the address in DR0
	Bit 2 (L1)	00 (20–21)	00 (22–23)	Execution	Triggers on instruction execution at the address in DR1
	Bit 2 (L1)	01 (20–21)	00 (22–23)	Write (1 byte)	Triggers on 1-byte write to the address in DR1
DR1	Bit 2 (L1)	11 (20–21)	11 (22–23)	Read/Write (4 bytes)	Triggers on 4-byte read or write to the address in DR1
	Bit 4 (L2)	00 (24–25)	00 (26–27)	Execution	Triggers on instruction execution at the address in DR2
	Bit 4 (L2)	01 (24–25)	00 (26–27)	Write (1 byte)	Triggers on 1-byte write to the address in DR2
	Bit 4 (L2)	11 (24–25)	11 (26–27)	Read/Write (4 bytes)	Triggers on 4-byte read or write to the address in DR2
DR2	Bit 6 (L3)	00 (28–29)	00 (30–31)	Execution	Triggers on instruction execution at the address in DR3
	Bit 6 (L3)	01 (28–29)	00 (30–31)	Write (1 byte)	Triggers on 1-byte write to the address in DR3

Breakpoint	Enable Bit (Lx)	RWx (Bits)	LENx (Bits)	Access Type	Description
	Bit 6 (L3)	11 (28–29)	11 (30–31)	Read/Write (4 bytes)	Triggers on 4-byte read or write to the address in DR3

WinDbg Essentials and Commands Used in This Module

Command	Brief Description
<code>!teb</code>	Displays the Thread Environment Block (TEB) of the current thread
<code>!peb</code>	Displays the Process Environment Block (PEB) of the current process
<code>dt <structure></code>	Displays the layout of a structure type
<code>dt <structure> <address></code>	Displays a structure instance at the specified address
<code>!process 0 0 powershell.exe</code>	Lists all active processes and filters for powershell.exe
<code>.process <address></code>	Sets the current process context to the specified address
<code>!dh</code>	Displays the PE header of a module
<code>!dh -i</code>	Displays the import table of a module
<code>!dh -e</code>	Displays the export table of a module
<code>!dh -s</code>	Displays the resources of a module
<code>!load</code>	Loads a debugger extension DLL
<code>?</code>	Evaluates a simple arithmetic or logical expression

Command	Brief Description
??	Evaluates an expression using C/C++ syntax
poi()	Dereferences a pointer and displays its value
!m	Lists all loaded modules in the process
!m m <module>	Filters the module list by module name
da <address>	Displays ASCII string at the given memory address
dd <address>	Displays memory in 4-byte (DWORD) format
dw <address>	Displays memory in 2-byte (WORD) format
db <address>	Displays memory in byte + ASCII format
dqs <address>	Displays memory in 8-byte (QWORD) format with symbol resolution
dqs poi(<address>)	Displays memory pointed to by an address in QWORD format
u <address>	Disassembles code starting at the specified address
.fnent <address>	Displays function entry information at a specified address
bp <address/symbol>	Sets a breakpoint at the given address or symbol
g	Resumes execution of the target
r <register>	Displays or modifies the value of a CPU register
~	Displays all threads in the process
~!s	Switches to thread 1 and displays its call stack
k	Displays the current thread's call stack
~*k	Displays call stacks for all threads



HTB ACADEMY CHEATSHEET

HTB ACADEMY CHEATSHEET